

Minimum Size Subarray Sum

Two approaches, visualized execution, and the core logic to remember

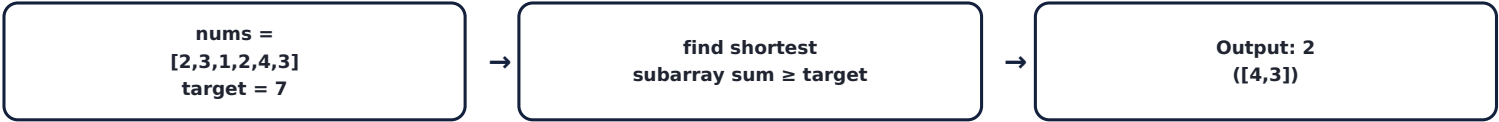
Brute Force · $O(n^2)$

Sliding Window · $O(n)$

Reference notes · `MinimumSizeSubArraySum.java`

Problem Statement

Given an array of positive integers `nums` and a positive integer `target`, return the minimal length of a contiguous subarray whose sum is $\geq$ `target`. Return `0` if no such subarray exists.



Approach Comparison

Approach	Time	Space	Core Idea
Brute Force	$O(n^2)$	$O(1)$	Try every starting point, grow until target is hit
Sliding Window	$O(n)$	$O(1)$	Expand right until valid, shrink left while still valid

Big picture: This is a *variable-size* sliding window, different from the fixed-size kind. The window grows and shrinks based on a condition ($\text{sum} \geq \text{target}$) instead of always staying at a fixed width — each pointer only ever moves forward, so every element is visited a constant number of times overall.

1 · Brute Force

$O(n^2)$ time

$O(1)$ space

For each
start index low



Grow window,
accumulate sum



sum $\geq$ target?
record length

```
// try every starting point, grow until target is hit
private static int usingBruteForce(int[] nums, int target) {
    int minWindow = Integer.MAX_VALUE;

    for (int low = 0; low < nums.length; low++) {
        int sum = 0;

        for (int high = low; high < nums.length; high++) {
            sum += nums[high];

            if (sum >= target) {
                int currentWindow = high - low + 1;
                minWindow = Math.min(minWindow, currentWindow);
            }
        }
    }

    return minWindow == Integer.MAX_VALUE ? 0 : minWindow;
}
```

Recall trick: "Try every starting point, grow until target is hit." Correct, but re-walks overlapping ranges for every low — $O(n^2)$ overall.

2 · Sliding Window (Variable Size)

O(n) time

O(1) space

Expand right:
sum += nums[high]



sum ≥ target?
record window



Shrink left while
still valid

```
// expand right until valid, shrink left while still valid
private static int usingSlidingWindow(int[] nums, int target) {
    int minWindow = Integer.MAX_VALUE;
    int sum = 0;
    int low = 0;
    int high = 0;

    while (high < nums.length) {
        sum += nums[high];
        high++;

        while (sum >= target) {
            int currentWindow = high - low;
            minWindow = Math.min(minWindow, currentWindow);

            sum = sum - nums[low];
            low++;
        }
    }
    return minWindow == Integer.MAX_VALUE ? 0 : minWindow;
}
```

Trace for nums = [2,3,1,2,4,3], target = 7:

high (after++)	sum	low	shrink?	window recorded	minWindow
1	2	0	no (2<7)	—	MAX
2	5	0	no (5<7)	—	MAX
3	6	0	no (6<7)	—	MAX
4	8	0→1	yes (8≥7)	len 4	4
5	10	1→3	yes (10≥7→6≥7)	len 4, then len 3	3
6	9	3→5	yes (9≥7→stop at 6<7)	len 3, then len 2	2

Recall trick: "Expand right until valid, then shrink left while still valid." Each element is added once (by high) and removed once (by low) across the entire run — that's what keeps it O(n) despite the nested loops.

TL;DR — For Future Me

What to remember

- **Best solution:** Variable-size sliding window, O(n) — "expand right until the sum is big enough, then shrink left as long as it stays big enough, recording the smallest window each time."
- **Brute force** is O(n²) — correct but re-walks overlapping ranges for every starting index.
- **Underlying pattern:** this is the *variable-size* sliding window template, different from the fixed-size one (like max-sum-subarray-of-size-k). The window grows and shrinks based on a condition rather than staying at a fixed width k. Each pointer only ever moves forward — that's the O(n) guarantee despite the nested loops.